SIMD Vector Processing Unit: Design and Verification of a Parameterized Multi-Lane Datapath in Verilog

Phase I Report — Team LANE BREAKERS

Dhanvin Vadlamudi
NST, Rishihood University
Enrolment No. 2401010150
dhanvin.vadlamudi2024@nst.rishihood.edu.in

Chaitanya Sai Meka
NST, Rishihood University
Enrolment No. 2401010275
meka.sai2024@nst.rishihood.edu.in

Jagruthi Pulumati
NST, Rishihood University
Enrolment No. 2401010196
jagruthi.pulumati2024@nst.rishihood.edu.in

Kasula Lalithendra
NST, Rishihood University
Enrolment No. 2401010216
kasula.lalithendra2024@nst.rishihood.edu.in

Polana Rakshita
NST, Rishihood University
Enrolment No. 2401020044
polana.rakshita2024@nst.rishihood.edu.in

Abstract—Single Instruction, Multiple Data (SIMD) execution is one of the most area- and energy-efficient ways to exploit data-level parallelism, and it underpins everything from classic vector supercomputers to today's CPU vector extensions and GPU shader cores. This report presents the Phase I design and verification of a parameterized SIMD Vector Processing Unit (VPU) written in synthesizable Verilog-2001. A single LANES parameter ($1, 2, 4$, or 8) replicates a 32-bit lane datapath, so one instruction updates every lane in the same clock cycle. The VPU supports nine lane-wise ALU operations (add, subtract, AND, OR, XOR, signed less-than, equality, and logical shifts), a scalar-broadcast operand mode, an 8-entry vector register file, an 8-entry scalar register file, and wide-port Harvard instruction and data memories. In Phase I we implemented all datapath and memory modules, integrated them into a demonstration top level, and verified them with self-checking Icarus Verilog testbenches, alongside a Python golden reference model. All 10 testbench runs (201 individual checks, including lane-count sweeps at 1, 4, and 8 lanes) pass, and simulation waveforms confirm that all four lanes of a vector add are written back in a single cycle. Phase II will add the instruction-sequencing control unit, run benchmark programs to measure speedup over a scalar baseline, and synthesize the design for a Xilinx Artix-7 FPGA.

Index Terms—SIMD, vector processor, data-level parallelism, Verilog, FPGA, hardware verification

I. Introduction

Many important workloads—image and signal processing, machine-learning inference, and scientific simulation—apply the same operation to long arrays of independent elements. This property, called data-level parallelism (DLP), can be exploited by a SIMD machine: the processor fetches and decodes one instruction and applies it to many data elements at once, amortizing the control overhead across all lanes [1], [2].

The aim of this project is to build such a machine from first principles at a scale small enough to understand completely: a parameterized SIMD VPU whose lane count can be changed without editing the RTL. The project lets us study, in real hardware description code, the main building blocks of a processor—register files, ALUs, operand multiplexing, Harvard memories, and FSM control—and the classic area-versus-performance trade-off of widening a datapath.

A. Objectives

1) Implement a parameterized VPU in synthesizable Verilog-2001 supporting LANES $\in \{1, 2, 4, 8\}$ from a single source.
2) Verify every module with self-checking testbenches and build a Python golden reference model. (Phase I — complete.)
3) Add a control unit that executes stored programs, and measure cycle-count speedup over the scalar (LANES $= 1$) baseline. (Phase II.)
4) Synthesize the design for Artix-7 and report resource use and maximum clock frequency per lane count. (Phase II.)

II. Literature Review

A. Flynn's Taxonomy

Flynn classified computer organizations by the number of concurrent instruction and data streams [1], and later analysed the effectiveness of each class [3]. In the SIMD class a single control unit broadcasts each instruction to many processing elements, each operating on its own data. Our VPU is a direct, minimal realization of this model: one shared alu_op drives all lane ALUs.

B. Vector Supercomputers

The Cray-1 [4] popularized the vector register architecture: vectors are loaded into large registers, operated on by pipelined functional units, and written back to memory. Hennessy and Patterson [2] describe how modern vector machines improve throughput by splitting each functional unit into multiple parallel lanes, and Asanović [5] showed that such lane-parallel vector designs remain effective in single-chip microprocessors. Our design adopts the vector-register and multi-lane organization, but processes a whole vector register in one cycle rather than streaming elements through a pipeline.

C. Packed SIMD Extensions and RISC-V Vectors

General-purpose processors added short "packed" SIMD units, beginning with Intel MMX [6], which treat a wide register as several narrower elements. Kozyrakis and Patterson [7] argued that true vector architectures scale better than fixed-width packed SIMD because software does not need to change when the hardware width changes. The RISC-V "V" extension [8] follows this vector-length-agnostic approach. Our LANES parameter mirrors the same idea at the hardware level: the same RTL and instruction set work at any lane count.

D. Soft Vector Processors on FPGAs

Several groups have built vector processors as soft cores on FPGAs. VIPERS [9] showed that a soft vector co-processor with a configurable number of lanes can deliver large speedups over a scalar soft CPU. VESPA [10] studied how performance and area scale as lanes are added, and VEGAS [11] replaced the vector register file with a scratchpad to reduce memory overhead. These works motivate our Phase II plan to synthesize the VPU at several lane counts and compare the LUT and flip-flop cost per lane against the cycle-count speedup.

E. Speedup Limits

Amdahl's law [12] bounds the achievable speedup by the fraction of work that cannot be parallelized. With L lanes, a purely data-parallel loop approaches an $L\times$ speedup, but scalar instructions for address arithmetic, loop counters, and branches do not shrink with L. We therefore expect sub-linear speedup in Phase II, and our benchmark plan is designed to quantify this gap.

III. Proposed Solution

A. Architecture Overview

The VPU is a non-pipelined, single-issue Harvard machine with separate instruction and data memories. Its datapath is shown in Fig. 1. Vector registers are stored as a single flat bus of LANES $\times$ 32 bits, with lane i occupying bits [i*32 +: 32]. Every lane receives the same opcode, so a single instruction performs LANES operations in parallel.

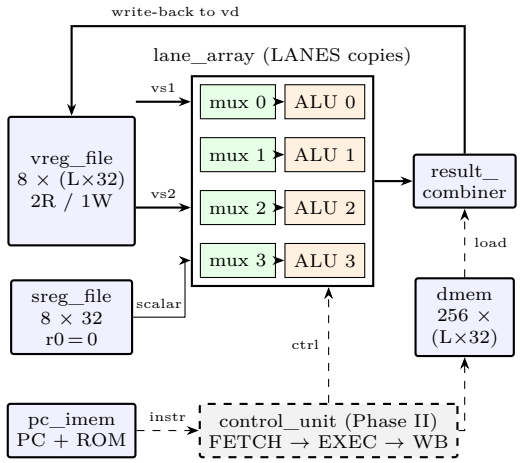


Fig. 1. VPU datapath (shown for LANES = 4). Each lane has its own operand-B multiplexer and ALU; all lanes share one opcode. Dashed blocks and paths are Phase II: the control unit is designed but not yet implemented, and dmem and pc_imem are implemented and unit-tested but not yet connected. In Phase I the control signals are driven by the testbench through vpu_demo_top.

TABLE I
RTL Module Summary (Phase I)

Module	Function
lane_alu	Combinational 32-bit ALU, 9 operations selected by a 4-bit alu_op
lane_mux	Selects operand B: the lane's own vs2 element, or a broadcast scalar
lane_array	generate loop creating LANES mux+ALU pairs
vreg_file	8 vector registers, 2 async read ports, 1 sync write port
sreg_file	8 scalar registers, r0 hard-wired to zero
result_combiner	Write-back mux: ALU result or memory load data
dmem	Data memory, one LANES×32-bit word per address
pc_imem	Program counter with branch support, 256-word instruction ROM
vpu_demo_top	Phase I integration of the full datapath

B. Module Design

Table I lists the RTL modules. All are parameterized by LANES and data width DW = 32.

Lane ALU. The ALU implements VADD, VSUB (32-bit wrap-around), VAND, VOR, VXOR, VSLT (signed compare, result 0 or 1), VEQ, and logical shifts VSLL/VSRL using the low 5 bits of operand B. It is purely combinational, so the result is ready in the same cycle as its inputs.

Scalar broadcast. When opb_sel = 1, every lane's multiplexer selects the same scalar register value, giving instructions such as VADD.S vd, vs1, rs2 that compute $vd[i] = vs1[i] + rs2$. This avoids having to fill a whole vector register with a repeated constant.

Register files. Both register files use asynchronous read and synchronous write, so an instruction can read its

operands, compute, and write its result on the next clock edge. Scalar register r0 always reads as zero (as in RISC-V); vector register v0 is a normal register.

Memories. dmem is addressed in lane-groups: address A holds LANES consecutive 32-bit elements, so a vector load or store moves a whole register in one access. pc_imem holds the program and updates the PC by +1 or to a branch target.

C. Instruction Set

Instructions are 32 bits wide: opcode[31:26], vd[25:23], vs1[22:20], vs2[19:17], opb_sel[16], and a 16-bit signed immediate. The ISA contains the nine vector ALU operations, vector load/store (VLD, VST), scalar SADDI and SLI for loop counters and addresses, BNE for loops, and HALT. The full encoding is in docs/isa.md.

D. Control Unit Design

The control unit (to be implemented in Phase II) is a three-state Moore FSM: FETCH → DECODE/EXECUTE → WRITEBACK. Vector ALU operations and VLD take three cycles; VST, BNE, and HALT take two. The state diagram and decode table are in docs/control_fsm.md.

E. Key Design Decisions

- Single parameter for width: changing LANES rescales the register file, ALU array, and memory width together, so speedup can be measured on identical RTL.
- Flat vector buses ([LANES*32-1:0]) keep the code Verilog-2001 compatible and synthesizable on any tool.
- Non-pipelined control keeps the design hazard-free, so cycle counts are exact and easy to reason about; pipelining is future work.
- Target device: Xilinx Artix-7 xc7a35tcpg236-1.

IV. Verification and Phase I Progress

A. Method

Each module has a self-checking testbench that drives inputs, compares outputs with expected values written into the testbench, and prints PASS or FAIL: expected=X actual=Y. Independently, a Python golden model (verif/golden.py) implements the same nine operations with 32-bit wrap-around and signed semantics; it has its own self-test and will generate the expected outputs for the Phase II benchmark programs. Test vectors include edge cases: 0, 1, −1, 0x7FFFFFFF, 0x80000000, overflow wrap-around, and shifts by 0 and 31. The lane_array testbench is compiled at LANES = 1, 4, and 8 and includes a lane-independence test: changing one lane's input must change only that lane's output. A single make test command runs the whole suite.

TABLE II
Phase I Testbench Results (Icarus Verilog)

Testbench	Scope	Checks	Result
golden.py	golden-model self-test	–	PASS
lane_alu_tb	9 ops + edge cases	37	PASS
lane_mux_tb	vector vs. scalar select	9	PASS
lane_array_tb	LANES = 1	7	PASS
lane_array_tb	LANES = 4	26	PASS
lane_array_tb	LANES = 8	50	PASS
vreg_file_tb	all 8 regs, 2 ports	17	PASS
sreg_file_tb	r0 = 0 invariant	14	PASS
dmem_tb	lane-group read/write	9	PASS
pc_imem_tb	reset, increment, branch	12	PASS
vpu_demo_top_tb	integrated datapath	20	PASS
Total (10 RTL testbench runs)		201	PASS

B. Results

Table II shows the output of make test under Icarus Verilog. All 201 checks pass.

The integrated testbench (vpu_demo_top_tb) loads two vectors into v0 and v1, then executes VADD, VSUB, VAND, a vector-vector VSLT, and an all-lane overflow case. Test T4 compares v0 against $v5 = [5, 5, 5, 5]$ with opb_sel = 0 and writes the result to v6. The lane_mux_tb and lane_array_tb tests exercise scalar broadcast with opb_sel = 1.

In the VADD test, with $v0 = [1, 2, 3, 4]$ and $v1 = [10, 11, 12, 13]$, VADD v2, v0, v1 produces $v2 = [11, 13, 15, 17]$, and all four lanes are written on the same clock edge. Fig. 2 shows the four lane values on alu_result before write-back and on result_out when the testbench reads back v2.

C. Phase I Status

Completed: all nine RTL modules, the integrated demonstration datapath, the golden model, ten self-checking testbench runs, the ISA and control-FSM specification, design documentation for every module, a Vivado synthesis script, and an automated Makefile flow (make test, make wave).

D. Plan for Phase II

1) Implement control_unit and a full vpu_top that fetches and executes programs from pc_imem.
2) Write benchmark programs (e.g. $C[i] = A[i] + B[i]$, scalar-broadcast scaling, and a compare/mask loop) and measure cycle counts at LANES = 1, 2, 4, 8 to compute speedup.
3) Run Vivado synthesis on Artix-7 at each lane count and report LUTs, flip-flops, and maximum frequency.
4) Analyse where speedup is close to linear and where scalar loop overhead makes it sub-linear, in light of Amdahl's law.
5) Stretch goals: VMUL using DSP48 slices, and a cross-lane reduction.

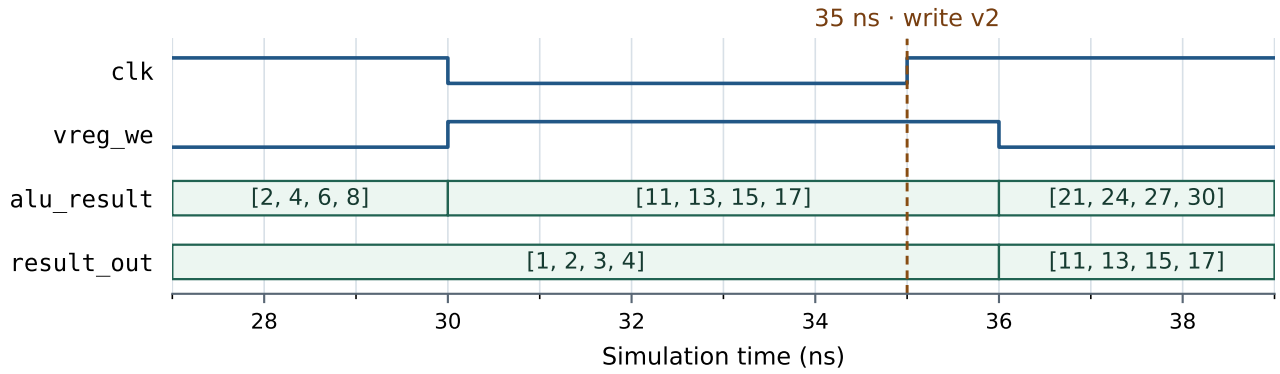


Fig. 2. Four-lane VADD waveform from the vpu_demo_top_tb simulation. Bus values are shown in lane order 0 through 3. With vreg_we high, the rising clock edge at 35 ns writes all four results to v2. The testbench selects v2 for readback at 36 ns.

V. Individual Contributions

All work is tracked in the team's GitHub repository; each member's commits can be seen in the version history.

- Dhanvin Vadlamudi (Datapath RTL): designed and implemented the 32-bit lane_alu, lane_mux, the parameterized lane_array, vreg_file, sreg_file, and result_combiner, with documentation for each module.
- Chaitanya Sai Meka (Report and Synthesis): wrote the IEEE report and literature review, and the Vivado non-project-mode synthesis script for Artix-7.
- Jagruthi Pulumati (Control and Memory): specified the ISA and instruction encoding, designed the control FSM, and implemented dmem and pc_imem with their testbenches and documentation.
- Kasula Lalithendra (Architecture and Integration): created the repository structure, project specification, interface contracts, block diagram, and design-decision log; integrated all modules in vpu_demo_top with its testbench; and built the Makefile test and waveform flow.
- Polana Rakshita (Verification): wrote the Python golden model, the testbenches for the ALU, multiplexer, lane array (1/4/8 lanes), and register files, and the verification plan, and captured the 4-lane waveform.

VI. Conclusion

Phase I delivers a complete, verified SIMD datapath: a single Verilog source that scales from one to eight 32-bit lanes and executes each vector instruction across all lanes in one cycle. Every module passes its self-checking testbench, covering edge cases and lane counts of 1, 4, and 8. Phase II will turn this datapath into a programmable processor and measure its speedup and FPGA cost as the number of lanes grows.

References

[1] M. J. Flynn, "Very high-speed computing systems," Proceedings of the IEEE, vol. 54, no. 12, pp. 1901–1909, 1966.

[2] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Morgan Kaufmann, 2017.

[3] M. J. Flynn, "Some computer organizations and their effectiveness," IEEE Transactions on Computers, vol. C-21, no. 9, pp. 948–960, 1972.

[4] R. M. Russell, "The CRAY-1 computer system," Communications of the ACM, vol. 21, no. 1, pp. 63–72, 1978.

[5] K. Asanović, "Vector microprocessors," Ph.D. dissertation, University of California, Berkeley, 1998.

[6] A. Peleg and U. Weiser, "MMX technology extension to the Intel architecture," IEEE Micro, vol. 16, no. 4, pp. 42–50, 1996.

[7] C. E. Kozyrakis and D. A. Patterson, "Scalable vector processors for embedded systems," IEEE Micro, vol. 23, no. 6, pp. 36–45, 2003.

[8] RISC-V "V" Vector Extension, Version 1.0, RISC-V International, 2021.

[9] J. Yu, G. Lemieux, and C. Eagleston, "Vector processing as a soft-core CPU accelerator," in Proc. 16th ACM/SIGDA Int. Symp. Field Programmable Gate Arrays (FPGA), 2008, pp. 222–232.

[10] P. Yiannacouras, J. G. Steffan, and J. Rose, "VESPA: Portable, scalable, and flexible FPGA-based vector processors," in Proc. Int. Conf. Compilers, Architectures and Synthesis for Embedded Systems (CASES), 2008, pp. 61–70.

[11] C. H. Chou, A. Severance, A. D. Brant, Z. Liu, S. Sant, and G. G. F. Lemieux, "VEGAS: Soft vector processor with scratchpad memory," in Proc. 19th ACM/SIGDA Int. Symp. Field Programmable Gate Arrays (FPGA), 2011, pp. 15–24.

[12] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," Proceedings of the AFIPS Spring Joint Computer Conference, pp. 483–485, 1967.